

Module M214

Application Cloud Native

B.1 Introduire Express et Node.js

Table des matières

1	Rappel : Concept des APIs REST	2
1.1	Définition d'une API	2
1.2	Architecture REST	2
1.3	Communication Client-Serveur	2
2	Rappel : Méthodes du Protocole HTTP	3
2.1	Les méthodes HTTP principales	3
2.2	Codes de statut HTTP	3
2.3	Structure d'une requête/réponse HTTP	3
3	L'Écosystème Node.js	4
3.1	Qu'est-ce que Node.js ?	4
3.2	Caractéristiques principales	4
3.3	Pourquoi Node.js pour les APIs ?	4
3.4	Qu'est-ce qu'Express.js ?	4
3.5	Node.js natif vs Express.js	5
4	Configuration de l'Environnement de Développement	6
4.1	Installation de Node.js	6
4.1.1	Téléchargement	6
4.1.2	Vérification de l'installation	6
4.2	NPM - Node Package Manager	6
4.2.1	Commandes NPM essentielles	6
4.3	Création d'un projet Node.js	6
4.4	Le fichier package.json	7
4.5	Nodemon	7
5	L'Essentiel de Node.js	9
5.1	Les Modules	9
5.1.1	Qu'est-ce qu'un module ?	9
5.1.2	Types de modules	9
5.1.3	Créer et exporter un module	9
5.1.4	Importer et utiliser un module	9
5.1.5	Modules Core de Node.js	10
5.2	Programmation Asynchrone	10
5.2.1	Pourquoi l'asynchrone ?	10
5.2.2	1. Callbacks	10
5.2.3	2. Promises	11
5.2.4	3. Async/Await	12
5.3	Gestion des Erreurs	12
5.3.1	Try/Catch pour le code synchrone	13
5.3.2	Gestion d'erreurs avec Promises	13
5.3.3	Gestion d'erreurs avec Async/Await	13
5.4	Gestion des Fichiers (Module fs)	13
5.4.1	Lecture de fichiers	13
5.4.2	Écriture de fichiers	14
5.4.3	Travailler avec JSON	14

1 Rappel : Concept des APIs REST

1.1 Définition d'une API

API - Application Programming Interface

Une **API** est un ensemble de définitions et de protocoles qui permet à deux applications de communiquer entre elles.

Analogie : L'API est comme un serveur dans un restaurant - vous (le client) passez votre commande, le serveur (API) transmet à la cuisine (serveur/base de données) et vous rapporte le plat (réponse).

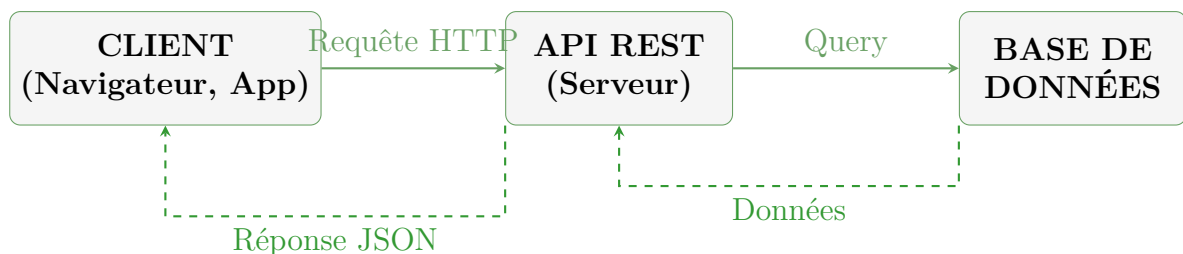
1.2 Architecture REST

REST - REpresentational State Transfer

REST est un style architectural pour concevoir des services web. Principes clés :

- **Client-Serveur** : Séparation des responsabilités
- **Sans état (Stateless)** : Chaque requête est indépendante
- **Ressources** : Tout est une ressource identifiée par une URL
- **Représentations** : JSON, XML pour représenter les données

1.3 Communication Client-Serveur



2 Rappel : Méthodes du Protocole HTTP

2.1 Les méthodes HTTP principales

GET	Read	Récupérer des données	GET /api/users
POST	Create	Créer une nouvelle ressource	POST /api/users
PUT	Update	Modifier une ressource (complète)	PUT /api/users/1
DELETE	Delete	Supprimer une ressource	DELETE /api/users/1

2.2 Codes de statut HTTP

2xx	Succès	200 OK, 201 Created, 204 No Content
4xx	Erreur Client	400 Bad Request, 404 Not Found
5xx	Erreur Serveur	500 Internal Server Error

2.3 Structure d'une requête/réponse HTTP

```

1 // REQUETE
2 GET /api/users/123 HTTP/1.1
3 Host: localhost:3000
4 Content-Type: application/json
5
6 // REPONSE
7 HTTP/1.1 200 OK
8 Content-Type: application/json
9
10 {
11     "id": 123,
12     "nom": "Ahmed",
13     "email": "ahmed@mail.com"
14 }
```

Listing 1 – Exemple de requête HTTP

3 L'Écosystème Node.js

3.1 Qu'est-ce que Node.js ?

Node.js

Node.js est un environnement d'exécution JavaScript côté serveur, construit sur le moteur V8 de Google Chrome.

Créé par : Ryan Dahl en 2009

Écrit en : C, C++, JavaScript

3.2 Caractéristiques principales

lightgray Caractéristique	Description
Single-threaded	Un seul thread principal (mais non-bloquant)
Event-driven	Basé sur les événements et callbacks
Non-bloquant (I/O)	Les opérations I/O n'attendent pas la fin pour continuer
NPM	Gestionnaire de packages avec +2 millions de packages
Cross-platform	Fonctionne sur Windows, macOS, Linux

3.3 Pourquoi Node.js pour les APIs ?

✓ Avantages

- **Performance** : Très rapide pour les opérations I/O
- **JavaScript partout** : Même langage côté client et serveur
- **Écosystème riche** : NPM avec des millions de packages
- **Temps réel** : Idéal pour WebSockets, chat, streaming
- **Scalabilité** : Gère efficacement de nombreuses connexions simultanées

3.4 Qu'est-ce qu'Express.js ?

Express.js

Express.js est un framework web minimaliste et flexible pour Node.js.

Slogan : *"Fast, unopinionated, minimalist web framework for Node.js"*

Express simplifie la création de serveurs web et d'APIs en fournissant :

- Système de routage simple
- Gestion des requêtes/réponses HTTP
- Support des middleware
- Intégration facile avec les bases de données

3.5 Node.js natif vs Express.js

```
1 const http = require('http');
2
3 const server = http.createServer((req, res) => {
4   if (req.url === '/' && req.method === 'GET') {
5     res.writeHead(200, {'Content-Type': 'text/plain'});
6     res.end('Hello World');
7   } else {
8     res.writeHead(404);
9     res.end('Not Found');
10  }
11 });
12
13 server.listen(3000, () => {
14   console.log('Serveur démarre sur le port 3000');
15 });
```

Listing 2 – Serveur HTTP avec Node.js NATIF

```
1 const express = require('express');
2 const app = express();
3
4 app.get('/', (req, res) => {
5   res.send('Hello World');
6 });
7
8 app.listen(3000, () => {
9   console.log('Serveur démarre sur le port 3000');
10 });
```

Listing 3 – Le même serveur avec Express.js

Conclusion

Express.js réduit considérablement la quantité de code nécessaire et fournit une API plus intuitive !

4 Configuration de l'Environnement de Développement

4.1 Installation de Node.js

4.1.1 Téléchargement

1. Aller sur <https://nodejs.org>
2. Télécharger la version **LTS** (Long Term Support)
3. Exécuter l'installateur et suivre les instructions

4.1.2 Vérification de l'installation

```
1 # Verifier la version de Node.js
2 $ node --version
3 v20.10.0
4
5 # Verifier la version de npm
6 $ npm --version
7 10.2.3
```

4.2 NPM - Node Package Manager

NPM

NPM est le gestionnaire de packages de Node.js. Il permet de :

- Installer des packages (bibliothèques) externes
- Gérer les dépendances d'un projet
- Partager son propre code avec la communauté

4.2.1 Commandes NPM essentielles

<code>npm init</code>	Initialiser un nouveau projet (crée package.json)
<code>npm init -y</code>	Initialiser avec les valeurs par défaut
<code>npm install <pkg></code>	Installer un package (dépendance)
<code>npm install <pkg> -save-dev</code>	Installer comme dépendance de développement
<code>npm install</code>	Installer toutes les dépendances du package.json
<code>npm uninstall <pkg></code>	Désinstaller un package
<code>npm run <script></code>	Exécuter un script défini dans package.json

4.3 Création d'un projet Node.js

```
1 # 1. Créer le dossier du projet
2 $ mkdir mon-projet
3 $ cd mon-projet
4
5 # 2. Initialiser le projet
6 $ npm init -y
7
8 # 3. Installer Express et Nodemon
9 $ npm install express
10 $ npm install --save-dev nodemon
```

4.4 Le fichier package.json

```
1 {
2   "name": "mon-projet",
3   "version": "1.0.0",
4   "description": "Mon premier projet Node.js",
5   "main": "index.js",
6   "scripts": {
7     "start": "node index.js",
8     "dev": "nodemon index.js"
9   },
10  "dependencies": {
11    "express": "^4.18.2"
12  },
13  "devDependencies": {
14    "nodemon": "^3.0.1"
15  }
16 }
```

Listing 4 – Structure du fichier package.json

Explication des champs

- **name** : Nom du projet
- **version** : Version du projet
- **main** : Point d'entrée de l'application
- **scripts** : Commandes personnalisées (npm run dev)
- **dependencies** : Packages nécessaires en production
- **devDependencies** : Packages nécessaires uniquement en développement

4.5 Nodemon

Nodemon

Nodemon est un outil qui redémarre automatiquement le serveur Node.js lorsque des fichiers sont modifiés. Indispensable en développement !


```
1 # Avec node (redemarrage manuel a chaque modification)
2 $ node index.js
3
4 # Avec nodemon (redemarrage automatique)
5 $ npm run dev
6 # ou directement
7 $ npx nodemon index.js
```

5 L'Essentiel de Node.js

5.1 Les Modules

5.1.1 Qu'est-ce qu'un module ?

Module

Un **module** est un fichier JavaScript qui encapsule du code réutilisable. Node.js utilise le système de modules **CommonJS**.

5.1.2 Types de modules

Core modules	Modules intégrés à Node.js	fs, http, path
Local modules	Modules créés par le développeur	./utils.js
Third-party	Modules installés via npm	express, lodash

5.1.3 Créer et exporter un module

```

1 // Définir des fonctions
2 function addition(a, b) {
3     return a + b;
4 }
5
6 function soustraction(a, b) {
7     return a - b;
8 }
9
10 function multiplication(a, b) {
11     return a * b;
12 }
13
14 // Exporter les fonctions
15 module.exports = {
16     addition,
17     soustraction,
18     multiplication
19 };
20
21 // OU exporter une par une
22 // module.exports.addition = addition;
23 // exports.soustraction = soustraction;
```

Listing 5 – math.js - Module personnalisé

5.1.4 Importer et utiliser un module

```

1 // Importer le module local
2 const math = require('./math');
```

```

3
4 // Utiliser les fonctions exportees
5 console.log(math.addition(5, 3));      // 8
6 console.log(math.soustraction(10, 4)); // 6
7 console.log(math.multiplication(2, 6)); // 12
8
9 // Import destructure
10 const { addition, multiplication } = require('./math');
11 console.log(addition(1, 2)); // 3

```

Listing 6 – app.js - Utilisation du module

5.1.5 Modules Core de Node.js

```

1 // Module fs (File System)
2 const fs = require('fs');
3
4 // Module path (chemins de fichiers)
5 const path = require('path');
6
7 // Module http (serveur web)
8 const http = require('http');
9
10 // Module os (informations systeme)
11 const os = require('os');
12 console.log(os.platform()); // 'win32', 'linux', 'darwin'

```

Listing 7 – Utilisation des modules core

5.2 Programmation Asynchrone

5.2.1 Pourquoi l'asynchrone ?

Node.js est **non-bloquant** : il n'attend pas la fin d'une opération longue (lecture fichier, requête réseau) pour continuer l'exécution.

SYNCHRONE (Bloquant) **ASYNCHRONE (Non-bloquant)**



5.2.2 1. Callbacks

La méthode traditionnelle : passer une fonction à exécuter quand l'opération est terminée.

```

1 const fs = require('fs');
2
3 // Lecture asynchrone avec callback

```

```

4 fs.readFile('fichier.txt', 'utf8', (err, data) => {
5     if (err) {
6         console.error('Erreur:', err);
7         return;
8     }
9     console.log('Contenu:', data);
10 });
11
12 console.log('Cette ligne s\'affiche AVANT le contenu du fichier !');

```

Listing 8 – Exemple avec callback

⚠ Callback Hell

Les callbacks imbriqués deviennent vite illisibles :

```

1 fs.readFile('a.txt', (err, a) => {
2     fs.readFile('b.txt', (err, b) => {
3         fs.readFile('c.txt', (err, c) => {
4             // Callback hell...
5         });
6     });
7 });

```

5.2.3 2. Promises

Les Promises représentent une valeur qui sera disponible dans le futur.

```

1 // Créer une Promise
2 const maPromise = new Promise((resolve, reject) => {
3     const succes = true;
4
5     if (succes) {
6         resolve('Operation reussie !');
7     } else {
8         reject('Erreur survenue');
9     }
10 });
11
12 // Utiliser la Promise
13 maPromise
14     .then(resultat => {
15         console.log(resultat); // 'Operation reussie !'
16     })
17     .catch(erreur => {
18         console.error(erreur);
19     });

```

Listing 9 – Création et utilisation de Promises

```

1 function etape1() {
2     return Promise(resolve => {
3         setTimeout(() => resolve('Etape 1 terminee'), 1000);

```

```
4     });
5 }
6
7 function etape2(message) {
8     return new Promise(resolve => {
9         setTimeout(() => resolve(message + ' -> Etape 2 terminee'), 1000);
10    });
11 }
12
13 // Chainage
14 etape1()
15     .then(result => etape2(result))
16     .then(finalResult => console.log(finalResult))
17     .catch(err => console.error(err));
18 // Affiche apres 2 secondes: "Etape 1 terminee -> Etape 2 terminee"
```

Listing 10 – Chaînage de Promises

5.2.4 3. Async/Await

Syntaxe moderne et plus lisible pour travailler avec les Promises.

```
1 // Fonction asynchrone
2 async function traitement() {
3     try {
4         const result1 = await etape1();
5         console.log(result1);
6
7         const result2 = await etape2(result1);
8         console.log(result2);
9
10        return result2;
11    } catch (err) {
12        console.error('Erreur:', err);
13    }
14 }
15
16 // Appeler la fonction
17 traitement();
```

Listing 11 – Async/Await - Syntaxe moderne

✓ Async/Await - Avantages

- Code plus lisible (ressemble au code synchrone)
- Gestion des erreurs avec try/catch
- Plus facile à déboguer

5.3 Gestion des Erreurs

5.3.1 Try/Catch pour le code synchrone

```
1 try {
2   const data = JSON.parse('json invalide');
3   console.log(data);
4 } catch (error) {
5   console.error('Erreur de parsing:', error.message);
6 }
7
8 console.log('Le programme continue...');
```

Listing 12 – Gestion d’erreurs synchrone

5.3.2 Gestion d’erreurs avec Promises

```
1 maPromise
2   .then(result => {
3     console.log(result);
4   })
5   .catch(error => {
6     console.error('Erreur:', error);
7   })
8   .finally(() => {
9     console.log('Execution terminee (succes ou erreur)');
10  });
```

Listing 13 – Gestion d’erreurs avec .catch()

5.3.3 Gestion d’erreurs avec Async/Await

```
1 async function lireFichier() {
2   try {
3     const fs = require('fs').promises;
4     const contenu = await fs.readFile('fichier.txt', 'utf8');
5     console.log(contenu);
6   } catch (error) {
7     if (error.code === 'ENOENT') {
8       console.error('Fichier non trouve');
9     } else {
10      console.error('Erreur:', error.message);
11    }
12  }
13 }
```

Listing 14 – Try/Catch avec Async/Await

5.4 Gestion des Fichiers (Module fs)

5.4.1 Lecture de fichiers

```
1 const fs = require('fs');
2
3 // 1. Methode synchrone (BLOQUANTE - a eviter)
4 const dataSync = fs.readFileSync('fichier.txt', 'utf8');
5 console.log(dataSync);
6
7 // 2. Methode avec callback
8 fs.readFile('fichier.txt', 'utf8', (err, data) => {
9     if (err) throw err;
10    console.log(data);
11 });
12
13 // 3. Methode avec Promises (RECOMMANDEE)
14 const fsPromises = require('fs').promises;
15
16 async function lire() {
17     const data = await fsPromises.readFile('fichier.txt', 'utf8')
18     ;
19     console.log(data);
20 }
21 lire();
```

Listing 15 – Lecture de fichiers - 3 méthodes

5.4.2 Écriture de fichiers

```
1 const fs = require('fs').promises;
2
3 async function ecrire() {
4     // Ecrire (ecrase le contenu existant)
5     await fs.writeFile('output.txt', 'Nouveau contenu');
6
7     // Ajouter a la fin du fichier
8     await fs.appendFile('output.txt', '\nLigne ajoutee');
9
10    console.log('Fichier ecrit avec succes');
11 }
12
13 ecrire();
```

Listing 16 – Écriture de fichiers

5.4.3 Travailler avec JSON

```
1 const fs = require('fs').promises;
2
3 // Lire un fichier JSON
4 async function lireJSON() {
5     const data = await fs.readFile('data.json', 'utf8');
6     const obj = JSON.parse(data);
7     return obj;
8 }
```

```
9
10 // Ecrire un fichier JSON
11 async function ecrireJSON(obj) {
12     const json = JSON.stringify(obj, null, 2); // null, 2 =
        indentation
13     await fs.writeFile('data.json', json);
14 }
15
16 // Utilisation
17 async function main() {
18     const users = await lireJSON();
19     users.push({ id: 3, nom: 'Nouveau' });
20     await ecrireJSON(users);
21 }
```

Listing 17 – Lecture et écriture de fichiers JSON